

GitHub Copilot for Cybersecurity Specialists

- Tim Warner
 - Microsoft MVP
 - Microsoft Certified Trainer
-

Speaker Notes:

FRAMER: Welcome to Lesson 2. In Lesson 1, we detected vulnerabilities with Copilot. Today we're implementing secure systems from first principles. We're not just writing secure code - we're using GitHub Copilot and GitHub Advanced Security to build authentication, encryption, API gateways, and zero-trust architectures that pass enterprise security reviews.

BULLETS:

The shift from Lesson 1 to Lesson 2 is moving from reactive to proactive security. Detection finds problems after they're written. Prevention eliminates problem classes before code hits production. GitHub Copilot excels at both, but today we're focusing on its ability to generate secure implementations that follow OWASP, NIST, and industry best practices automatically.

Over the next 40 minutes, we're implementing security protocols using GitHub Copilot's deep knowledge of security patterns combined with GitHub Advanced Security's policy enforcement. These aren't theoretical examples - this is production infrastructure that passes SOC 2 audits and enterprise security reviews.

All code samples, infrastructure-as-code templates, and Copilot prompts are at timw.info/copilot-security/lesson-02. You'll get working OAuth servers, encryption libraries, API gateway middleware, and Terraform templates you can deploy today.

PRO TIP: The most powerful thing about GitHub Copilot for security isn't that it writes secure code faster - it's that it knows security patterns you might not. OAuth 2.0 with PKCE, AES-256-GCM with proper IV generation, JWT validation with clock skew tolerance. These aren't obscure details - they're the difference between secure and broken implementations. Copilot gets them right automatically.

Learning objectives

- Generate compliant infrastructure-as-code templates and security baselines
- Automate CIS and NIST benchmark verification scripts
- Build STIG compliance validation and auto-remediation tools
- Automate security documentation, audit logs, and incident response playbooks

Speaker Notes:

FRAMER: Compliance doesn't have to be the soul-crushing paperwork exercise we've all experienced. When you automate compliance checks, security baselines, and incident response procedures, compliance becomes continuous validation instead of quarterly panic. This is where Copilot transforms from a coding assistant into a compliance force multiplier.

BULLETS:

- Generate compliant infrastructure-as-code templates and security baselines - We'll use Copilot to create ARM templates, Terraform configurations, and Azure Bicep files that are secure by default. These templates include encryption settings, network security groups, RBAC policies, and logging configurations that meet compliance requirements from day one. Contoso reduced their infrastructure provisioning audit findings by 90% using hardened IaC templates.*
- Automate CIS and NIST benchmark verification scripts - Manual compliance checking is tedious and error-prone. We'll build PowerShell and Python scripts that automatically validate CIS benchmarks and NIST 800-53 controls across your infrastructure. These scripts run continuously, catching configuration drift before auditors do. Fabrikam runs these checks hourly on all production servers.*
- Build STIG compliance validation and auto-remediation tools - STIGs (Security Technical Implementation Guides) are Department of Defense security standards that commercial organizations also adopt. We'll create scripts that not only detect STIG violations but automatically remediate them. No more manual server hardening spreadsheets. Adventure Works uses these tools to maintain STIG compliance on 500+ Windows servers.*
- Automate security documentation, audit logs, and incident response playbooks - Auditors love documentation, but nobody loves writing it. We'll use Copilot to generate security architecture diagrams, control matrices, and incident response playbooks from your existing infrastructure and code. When you change your infrastructure, the documentation updates automatically. Tailwind Traders saved 40 hours per audit cycle using automated documentation.*

PRO TIP: Start with the compliance requirements that cause you the most pain. If you dread PCI-DSS audits, automate PCI compliance checks first. If SOC 2 Type II eats your time, automate SOC 2 evidence collection. Target your biggest time sink and let Copilot help you eliminate the drudgery.

I've seen compliance teams reclaim 60% of their time using this approach.

Generate IaC security templates with Copilot

- Infrastructure-as-code embeds security in provisioning
- Use secure-by-default templates for Azure, AWS, and GCP
- Copilot generates hardened configurations from requirements
- Version control compliance as code

Speaker Notes:

FRAMER: Here's the beauty of infrastructure-as-code: when you provision a server with a secure template, every server gets the same security baseline. No more "oops I forgot to enable encryption" or "I thought someone else configured the firewall." Security becomes deterministic.

BULLETS:

- Infrastructure-as-code embeds security in provisioning - Traditional infrastructure means manually clicking through Azure Portal or AWS Console, praying you remember all the security settings. IaC means your security requirements are expressed as code that executes automatically. When you provision a new Azure SQL database, the ARM template enforces TDE encryption, auditing, threat detection, and firewall rules without any manual steps.*
- Use secure-by-default templates for Azure, AWS, and GCP - Each cloud provider has security best practices that nobody remembers completely. We'll prompt Copilot with "create an Azure SQL database template that meets CIS Azure Foundations Benchmark requirements" and it generates a template with all required security controls. Wide World Importers built a library of 40 secure templates covering their entire Azure footprint.*
- Copilot generates hardened configurations from requirements - This is where it gets powerful. You can describe your security requirements in plain English and Copilot translates them to IaC. "Create an Azure VM that's HIPAA compliant with disk encryption, no public IP, connected to a hub-spoke network, and deployed to availability zones" becomes 300 lines of Bicep that does exactly that. The translation from requirements to implementation happens in seconds.*
- Version control compliance as code - Your IaC templates go in Git alongside your application code. When a new CIS benchmark version releases, you update your templates and deploy the changes. Every deployment after that automatically meets the new requirements. Northwind tracks their compliance posture through Git history - they can show auditors exactly when each control was implemented and by whom.*

PRO TIP: Build a template library that maps to compliance frameworks. Create a directory structure like /templates/cis-azure, /templates/nist-800-53, /templates/hipaa. When audit time comes, you point auditors to the templates and the deployment logs proving they were used. This approach transformed compliance from "prove we did it" to "here's the code that does it automatically."

Demo: ARM and Terraform hardening

- Create Azure SQL template with comprehensive security controls
- Generate Terraform configuration for hardened EC2 instances
- Add compliance validation tests to IaC pipelines
- Build reusable modules for common secure patterns

Speaker Notes:

FRAMER: Let's get hands-on with infrastructure hardening. We'll create ARM templates and Terraform configurations that implement defense-in-depth from the infrastructure layer up. Watch how Copilot helps us encode security requirements that would take hours to research and implement manually.

BULLETS:

- *Create Azure SQL template with comprehensive security controls - The demo shows an ARM template that provisions Azure SQL with TDE encryption, Microsoft Defender for SQL, auditing to Log Analytics, private endpoints instead of public access, and automated vulnerability assessments. We use Copilot to generate this by prompting with our security requirements. The template is about 400 lines of JSON that would take 3 hours to write manually but takes 10 minutes with Copilot.*
- *Generate Terraform configuration for hardened EC2 instances - We'll create a Terraform module for EC2 instances that implements IMDSv2, encrypted EBS volumes, Systems Manager Session Manager instead of SSH, CloudWatch detailed monitoring, and security group rules following least privilege. Contoso uses this module for all EC2 deployments, ensuring every instance starts secure.*
- *Add compliance validation tests to IaC pipelines - Here's the key insight: test your infrastructure code before deployment. We'll use tools like Checkov and tfsec to scan Terraform for security violations. If the template tries to create an S3 bucket without encryption, the pipeline fails before anything deploys. Fabrikam catches 90% of misconfigurations during pull request review, not in production.*
- *Build reusable modules for common secure patterns - Don't reinvent security for each deployment. Create Terraform modules or ARM template specs for patterns like "secure storage account," "hardened VM," or "compliant SQL database." Teams can consume these modules knowing they're secure by design. Tailwind Traders maintains 30 secure modules that cover their entire infrastructure footprint.*

PRO TIP: Use Copilot to add inline comments explaining WHY each security control exists. In your ARM template, add comments like "TDE encryption required by CIS Azure Benchmark 4.1.1" or "Private endpoints prevent data exfiltration per NIST 800-53 SC-7." When new team members read the templates, they understand the security reasoning. This turns IaC into security documentation.

Automate compliance benchmark checks

- CIS benchmarks define security baselines for systems and applications
- NIST 800-53 provides comprehensive security control framework
- Automated checking scales better than manual audits
- Continuous compliance replaces point-in-time assessments

Speaker Notes:

FRAMER: Compliance benchmarks like CIS and NIST are massive documents - hundreds of pages of controls and settings. Nobody can memorize them all. Automated checking means you encode the requirements once and validate them continuously.

BULLETS:

• CIS benchmarks define security baselines for systems and applications - CIS (Center for Internet Security) publishes security configuration guides for operating systems, databases, cloud platforms, and applications. Each benchmark contains specific settings like "disable SMBv1" or "require TLS 1.2 minimum." These are community-vetted recommendations that represent industry best practices. Adventure Works uses CIS benchmarks as their minimum security baseline across all systems.

• NIST 800-53 provides comprehensive security control framework - NIST 800-53 is the US government's security control catalog, but commercial organizations use it too because it's thorough and well-structured. It covers everything from access control to incident response to security assessments. Unlike CIS benchmarks which are prescriptive, NIST 800-53 is outcome-focused - it tells you what to achieve but not exactly how. Wide World Importers maps their security program to NIST 800-53, demonstrating control coverage to customers and auditors.

• Automated checking scales better than manual audits - Manual compliance checking is like manually testing code - it works for one server but breaks down at scale. If you have 500 servers, manually checking 200 CIS benchmark items means 100,000 individual checks. Automated scripts do this in minutes and can run hourly. Northwind reduced compliance checking from 40 hours per quarter to 15 minutes per day using automation.

• Continuous compliance replaces point-in-time assessments - Traditional compliance means an annual audit where you scramble to prove you were compliant for the past year. Continuous compliance means you validate every day and have proof. When configuration drift happens, you know immediately instead of discovering it during audit. Fabrikam shows auditors real-time compliance dashboards that are always current.

PRO TIP: Don't try to automate every control on day one. Start with the controls that fail most often in your environment. In my experience, these are usually password policies, logging configurations, and encryption settings. Automate those first, get them to 100% compliance, then expand to other controls. Progressive automation beats overwhelming yourself.

Demo: CIS and NIST verification scripts

- Generate PowerShell script for CIS Windows Server benchmark
- Create Python tool for NIST 800-53 control validation
- Produce compliance reports in auditor-friendly formats
- Schedule automated compliance scans and alerting

Speaker Notes:

FRAMER: Now watch how we use Copilot to build compliance automation tools. We're going to create scripts that validate CIS benchmarks and NIST controls, then generate reports that auditors actually want to see.

BULLETS:

- Generate PowerShell script for CIS Windows Server benchmark - The demo shows a PowerShell script that checks 50+ CIS Windows Server 2022 controls. For each control, we test the actual configuration against the CIS requirement. "Is password minimum length set to 14 characters? Is SMBv1 disabled? Is Windows Firewall enabled?" The script outputs a compliance matrix showing pass/fail for each control. Contoso runs this script on every Windows server hourly.*
- Create Python tool for NIST 800-53 control validation - We'll use Copilot to generate Python code that validates NIST 800-53 controls in Azure. For example, control AC-6 (Least Privilege) maps to checking RBAC assignments for overly permissive roles. Control AU-2 (Audit Events) maps to verifying diagnostic settings on all resources. The tool queries Azure Resource Graph to validate these controls programmatically. Tailwind Traders extended this tool to cover 150 NIST controls.*
- Produce compliance reports in auditor-friendly formats - Auditors want Excel spreadsheets and PDF reports, not JSON output. The demo shows how to generate compliance reports in multiple formats. The Excel report has one tab per control family with pass/fail status, evidence links, and remediation recommendations. The PDF summary shows executive-level compliance percentages with trend graphs. Adventure Works built a report template that maps directly to their SOC 2 control requirements.*
- Schedule automated compliance scans and alerting - Compliance checking should be continuous, not quarterly. We'll use Azure Automation or GitHub Actions to schedule these scripts. When a control fails, send an alert to the security team with remediation instructions. When compliance drops below 95%, escalate to management. Wide World Importers maintains 97% compliance because they fix issues within hours instead of waiting for the next audit.*

PRO TIP: Store your compliance scan results in a database with timestamps. This gives you audit trails showing compliance over time. When an auditor asks "were you compliant on June 15th?" you can show them the exact scan results from that date. Fabrikam uses Azure Log Analytics to store compliance results, making them queryable with KQL for any historical date.

STIG validation and auto-remediation

- STIGs (Security Technical Implementation Guides) from DISA
- Automated scanning detects STIG violations across systems
- Auto-remediation fixes common misconfigurations safely
- Maintain compliance through configuration management

Speaker Notes:

FRAMER: STIGs are the Department of Defense's security standards, and they're intense. Hundreds of highly specific controls covering operating systems, applications, and network devices. Manual STIG compliance is painful. Automated STIG compliance is manageable.

BULLETS:

- STIGs (Security Technical Implementation Guides) from DISA - DISA (Defense Information Systems Agency) publishes STIGs for almost every technology: Windows, Linux, SQL Server, Oracle, network devices, you name it. Each STIG contains specific checks like registry settings, file permissions, and service configurations. Organizations pursuing DoD contracts must meet STIGs. But many commercial organizations also use STIGs because they represent thorough security hardening. Northwind adopted Windows Server STIGs even though they're not DoD contractors because the controls are comprehensive.*
- Automated scanning detects STIG violations across systems - We'll use tools like SCAP (Security Content Automation Protocol) scanners and custom PowerShell scripts to check STIG compliance. These tools compare actual system configurations against STIG requirements and identify violations. The scan produces a report showing findings categorized by severity: CAT I (high), CAT II (medium), CAT III (low). Contoso scans their servers weekly and tracks STIG compliance as a KPI.*
- Auto-remediation fixes common misconfigurations safely - Here's where it gets powerful. Some STIG violations can be automatically fixed: disable unnecessary services, set registry keys, configure file permissions. We'll build remediation scripts that fix CAT II and CAT III findings automatically after human review of CAT I findings. Adventure Works auto-remediates 60% of STIG violations, leaving security teams to focus on complex findings.*
- Maintain compliance through configuration management - STIG compliance isn't one-time - it requires continuous maintenance. We'll use tools like DSC (Desired State Configuration) or Ansible to enforce STIG settings continuously. If someone manually changes a configuration that violates a STIG requirement, DSC automatically reverts it. Fabrikam uses Azure Automate to apply STIG settings to all VMs with drift detection and automatic correction.*

PRO TIP: Start with Windows Server STIGs if you're new to this. Windows Server STIG has about 200 controls, which sounds overwhelming but many are straightforward. Use DISA's STIG Viewer tool to understand each control, then build your automation progressively. I recommend automating 20 controls per sprint. After two sprints, you'll have 40 controls automated and a clear path to 100%.

Demo: STIG server hardening workflows

- Scan Windows Server for STIG compliance gaps
- Generate auto-remediation scripts with Copilot
- Apply STIG baselines using PowerShell DSC
- Validate remediation and document evidence

Speaker Notes:

FRAMER: Let's dive into practical STIG automation. We're going to scan a Windows Server, identify violations, generate remediation code, and apply it safely. Watch how Copilot helps us go from STIG requirements to running code.

BULLETS:

- Scan Windows Server for STIG compliance gaps - The demo shows using PowerSTIG (a PowerShell module for STIG automation) to scan a Windows Server 2022 against current STIG requirements. The scan takes about 5 minutes and produces a detailed report showing 180 checks: 150 pass, 20 fail, 10 not applicable. The failures include things like "guest account not disabled" and "maximum password age not configured." This gives us our remediation roadmap.*
- Generate auto-remediation scripts with Copilot - Here's where Copilot shines. We paste the STIG finding text into Copilot Chat and prompt "generate PowerShell code to remediate this STIG requirement." It produces idempotent scripts that safely apply the required configuration. For example, for "V-220726: Guest account must be disabled," Copilot generates code that checks if the account exists and disables it only if needed. Wide World Importers built a library of 200 STIG remediation scripts this way.*
- Apply STIG baselines using PowerShell DSC - PowerShell Desired State Configuration is perfect for STIG enforcement. We define the desired state (STIG-compliant configuration) and DSC ensures systems maintain that state. The demo shows a DSC configuration that applies 50 STIG settings to a server. If someone manually changes a setting that violates a STIG requirement, DSC automatically reverts it on the next consistency check (default: every 15 minutes).*
- Validate remediation and document evidence - After applying remediation, scan again to verify fixes worked. The demo shows comparing before and after scans: violations dropped from 20 to 2. The remaining 2 require manual intervention (organizational policy decisions). We export the after-scan report as evidence for auditors. Tailwind Traders maintains a Git repository of scan reports timestamped and signed, providing auditable proof of continuous STIG compliance.*

PRO TIP: Test STIG remediation in dev before production. Some STIG settings can break applications - for example, disabling TLS 1.0 might break legacy integrations. Build a test environment, apply STIG automation, and validate that applications still work. Then roll to production with confidence. I've seen organizations break production by applying untested STIGs. Don't be that organization.

Security documentation automation

- Documentation is evidence of security controls
- Generate architecture diagrams from infrastructure code
- Auto-create control matrices and policy documents
- Maintain living documentation that updates with code

Speaker Notes:

FRAMER: Real talk: nobody likes writing security documentation, but everybody needs it. Auditors need it, customers need it, new team members need it. The secret is automating documentation generation so it stays accurate without manual effort.

BULLETS:

- Documentation is evidence of security controls - Auditors don't take your word that you have security controls - they want documented evidence. Architecture diagrams showing network segmentation, control matrices mapping requirements to implementations, policy documents describing security procedures. Without documentation, you can't prove compliance even if you're technically compliant. Contoso learned this the hard way when an audit failed despite having excellent security because they couldn't document it.*
- Generate architecture diagrams from infrastructure code - Your Terraform and ARM templates describe your infrastructure precisely. We'll use Copilot to convert IaC into architecture diagrams automatically. Prompt it with "analyze this Terraform configuration and create a network diagram showing VPCs, subnets, security groups, and resource placement." You get a Mermaid diagram or Graphviz output that becomes your architecture documentation. When infrastructure changes, regenerate the diagram. Always accurate, never stale.*
- Auto-create control matrices and policy documents - Control matrices map compliance requirements to implementations. For example, "NIST 800-53 AC-2 Account Management is implemented by Azure AD with MFA and conditional access policies configured in Terraform module az-ad-security." We'll use Copilot to scan our codebase and generate these mappings automatically. Wide World Importers generates their entire SOC 2 control matrix from code annotations.*
- Maintain living documentation that updates with code - Traditional documentation gets written once and never updated. Living documentation is generated from code on every commit. When you add a new security control, the documentation automatically reflects it. When you remove a deprecated service, it disappears from diagrams. Fabrikam generates their security documentation in CI/CD pipeline and publishes it to an internal wiki automatically.*

PRO TIP: Use documentation-as-code principles. Store documentation templates in Git alongside your application code. When infrastructure changes, update documentation in the same pull request. Require documentation updates as part of code review. Adventure Works rejects PRs that add infrastructure without updating architecture docs. This cultural discipline keeps documentation accurate.

Demo: Incident response playbooks

- Generate IR playbooks for common security incidents
- Automate evidence collection and log aggregation
- Create runbooks for containment and remediation
- Integrate IR workflows with SIEM and ticketing systems

Speaker Notes:

FRAMER: When a security incident happens, you don't have time to figure out what to do. Incident response playbooks provide step-by-step procedures that your team follows during high-stress situations. Copilot can help generate these playbooks from your infrastructure and security tooling.

BULLETS:

- Generate IR playbooks for common security incidents - The demo shows using Copilot to create playbooks for ransomware, data breach, account compromise, and DDoS attacks. Each playbook has sections for detection, containment, eradication, recovery, and lessons learned. We prompt Copilot with "create an incident response playbook for ransomware in an Azure environment with steps for isolation, backup recovery, and threat hunting." The output is a detailed procedure that's specific to our technology stack. Northwind maintains 20 incident-specific playbooks.*
- Automate evidence collection and log aggregation - When an incident is detected, you need to collect evidence fast before attackers cover their tracks. We'll create PowerShell scripts that automatically gather logs from Azure Monitor, Azure AD sign-ins, Network Watcher flow logs, and VM diagnostics. The script packages everything into a timestamped archive with chain-of-custody documentation. Contoso uses this script to collect evidence within minutes of incident detection.*
- Create runbooks for containment and remediation - Containment actions need to be executed precisely and quickly. We'll build Azure Automation runbooks that isolate compromised VMs, revoke user sessions, block malicious IPs, and disable compromised accounts. These runbooks can be triggered manually or automatically by SIEM alerts. Tailwind Traders reduced their mean time to contain incidents from 2 hours to 15 minutes using automated runbooks.*
- Integrate IR workflows with SIEM and ticketing systems - Incidents generate multiple workflows: SIEM creates an alert, creates a ticket in ServiceNow, notifies the security team in Slack, triggers evidence collection scripts, and updates a status dashboard. The demo shows a GitHub Actions workflow that orchestrates these steps. When Microsoft Sentinel fires an alert for suspicious activity, the workflow automatically kicks off IR procedures. Fabrikam handles 90% of low-severity incidents with zero manual intervention.*

PRO TIP: Practice incident response regularly with tabletop exercises and simulations. Having playbooks is great, but knowing they work is better. Schedule quarterly IR drills where you simulate ransomware or data breach scenarios and follow your playbooks. Time how long each step takes and identify bottlenecks. Wide World Importers runs quarterly IR simulations and updates their playbooks based on lessons learned. When a real incident happened, their practiced response was flawless.

Key takeaways

- Automate compliance checking to replace manual audits
- Infrastructure-as-code embeds security controls in provisioning
- STIG and benchmark automation maintains continuous compliance
- Security documentation should be generated from code, not written manually

Speaker Notes:

FRAMER: Compliance and configuration management are the foundation of security operations. When you automate these processes, security becomes sustainable instead of heroic. Manual compliance doesn't scale, but automated compliance does.

BULLETS:

- Automate compliance checking to replace manual audits - Point-in-time audits tell you if you were compliant last Tuesday. Automated compliance tells you if you're compliant right now. The shift from periodic assessment to continuous validation fundamentally changes how you approach security. Contoso maintains 98% compliance because they detect and fix violations within hours instead of discovering them during annual audits.*
- Infrastructure-as-code embeds security controls in provisioning - Every server, database, and storage account gets the same security baseline because it's encoded in your IaC templates. No more configuration drift, no more "I forgot to enable encryption," no more manual hardening. Security becomes deterministic and auditable. Fabrikam eliminated 85% of infrastructure security findings by switching to hardened IaC templates.*
- STIG and benchmark automation maintains continuous compliance - Manual compliance checking doesn't scale past about 20 servers. Automated checking scales to thousands. CIS, NIST, and STIG compliance becomes continuous validation with automated remediation instead of quarterly panic with manual fixes. Adventure Works maintains STIG compliance on 500+ servers with a team of 3 using automation.*
- Security documentation should be generated from code, not written manually - Your code is the source of truth about your infrastructure and security controls. Documentation generated from code is always accurate and never stale. When code changes, documentation updates automatically. Wide World Importers generates 80% of their security documentation from code annotations and IaC templates.*

PRO TIP: Think of compliance automation as infrastructure. You build it once and maintain it like any other system. Your compliance scripts, IaC templates, and documentation generators are code that needs version control, testing, and updates. Treat compliance automation with the same engineering rigor you apply to your applications.

Next steps and course wrap-up

- Apply hardened IaC templates to one environment
- Automate one compliance framework that matters to your organization
- Build incident response playbooks for your top 3 threats
- Share your Copilot security patterns with the community

Speaker Notes:

FRAMER: We've covered a lot in this course - from vulnerability detection to security protocols to automated testing to code review to compliance. The key is starting small and building incrementally. Don't try to automate everything at once.

BULLETS:

- Apply hardened IaC templates to one environment - Pick your dev environment and deploy hardened infrastructure templates. Learn what works and what needs tuning. Validate that applications still function with the added security controls. Once you're confident, promote the templates to production. Contoso spent one sprint hardening their dev environment before rolling to production, catching configuration issues that would have caused outages.*
- Automate one compliance framework that matters to your organization - If you're PCI-DSS regulated, start there. If you need SOC 2, start there. If you're pursuing FedRAMP, start with NIST 800-53. Focus your automation efforts on the framework that creates the most work for your team. Fabrikam started with CIS Azure benchmarks because that's what their customers asked about most. One quarter later, they had automated 80% of CIS checks.*
- Build incident response playbooks for your top 3 threats - Identify the security incidents you're most likely to face. For most organizations, that's ransomware, account compromise, and data breach. Build detailed playbooks for these scenarios with automated evidence collection and containment procedures. Practice them quarterly. Northwind built ransomware IR playbooks and when they got hit 6 months later, their prepared response limited damage to one isolated VM.*
- Share your Copilot security patterns with the community - The patterns and prompts you develop are valuable to other security professionals. Share what works on GitHub, blog about your experiences, present at conferences. The security community benefits when we share knowledge. I've put all the code examples from this course at timw.info/copilot-security. Fork it, improve it, and share your enhancements.*

PRO TIP: This is the end of our course, but it's the beginning of your journey using Copilot for security work. Stay curious, keep experimenting, and remember that AI tools are force multipliers for security teams, not replacements. The combination of your security expertise plus Copilot's pattern

recognition creates something more powerful than either alone. Now go automate something.